

Python: Data Structures – **dictionary, tuple, set**

字典 Python's built-in “Database”

Cheng-Te Li (李政德)

chengte@mail.ncku.edu.tw

Dept. of Statistics, NCKU



Course Info (12/24)

- Programming exam 3: 12/31
- Final exam: 1/7
- Remind: Good news for you 😊
 - Select 2 highest from 3 programming exams
 - Select 4 highest from 5 HWs (HW1-HW5)
- No worry about your score (except for the last ones)
- **HW5** is out, Deadline: **1/10** 23:59 (Fri)
 - HW5 is significantly important for Programming exam 3
 - We will show you partial sample code for HW5
- Today
 - Lecture 8: File processing
 - Lecture 9: dictionary, tuple, set
- 填寫教學反應調查 <http://tfcfd.acad.ncku.edu.tw/ncku/tfcfd/>

Course Info: Quiz 3

- **12/31** (Tuesday): Programming Exam 3 (**Quiz 3**)
(第3次上機程式考試) for each student (not each team)
 - Start from **09:10am**
 - Coverage: Lectures till **12/24**
 - **Data is the same as HW5**
 - **Problems are similar to HW5**

***** Select two highest scores from Quiz 1, 2 and 3 *****

- Rules
 - Open book, slides, and paper-based materials
 - NO Internet (except for submission on Moodle)
 - You can use either PC here or your own laptop

Course Info: Final Exam (20%)

- **2020/01/07** (Tuesday): Final Exam (期末考)
 - Do not allow no-show (不接受補考)
- **Two Rooms: 大一教室 & 電腦教室 (will announce soon)**
- 09:10am – 12:00pm (寫完交卷即可離開)
- 紙本作答，題型類似期中考
NO Open book, NO slides, NO paper materials
- NO Computer, NO Internet
- **Cover all Lectures**
 - Lecture 1-9 (till 12/24), Homework 1-5, Quiz 1-3
- **Do NOT forget midterm problems**

Grading

10 Items		Percentage
Homework (team)	5 → 4 HWs	31% (1% HW0, 7.5% / HW)
Midterm Exam		20%
Final Exam		20%
Programming Exams	3 → 2 PEs	30% (15% each)
Participation (Ask Questions in TA Hours)		4%

Total

105%

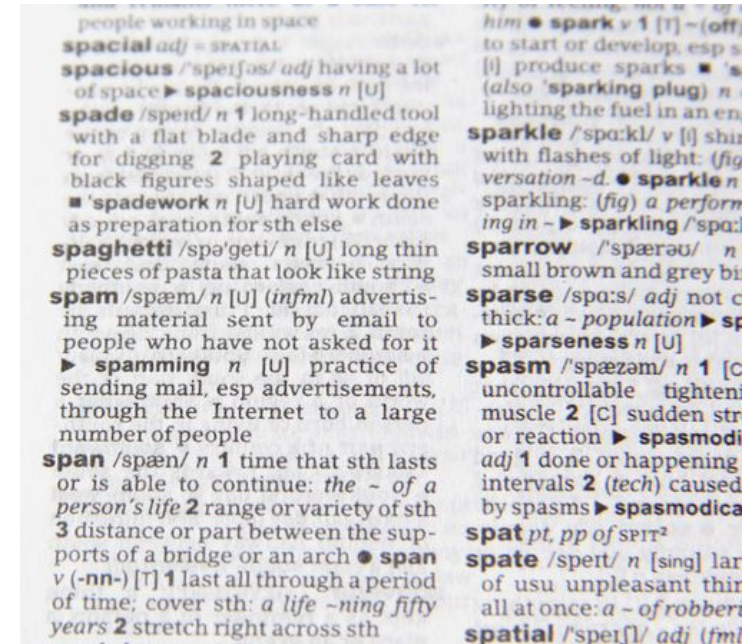
If your total score > 100%,
the final score will be 100%

Python Data Structures

- **Sequence: Ordered set of objects**
 - **Tuple ()**: sequence of values into a single object, may contain different data types, immutable
 - **List []**: sequence of values into a single object, may contain different data types, mutable
 - **String “ ”**: sequence of characters into a single object, contain only characters, immutable
- **Set { }**: unordered collection of objects
- **Dictionary { }**: collection of **unordered key-value** pairs

Dictionary

- Dictionary: a reference source in print or electronic form containing words usually alphabetically arranged along with information about their forms, functions, pronunciations, etymologies, meanings, and syntactical and idiomatic uses



Cat

Scientific name: feliscatus.

A cat is a small, meat-eating animal that is often kept as a pet.

word → definition & picture



Python

Scientific name: genus: python.

Pythons are non-venomous snakes that usually kill their prey by wrapping around them and suffocating them



Dictionary

- Dictionary – a list of pairs (**word** → **definition**)
- Python Dictionary – a list of pairs (**key** : **value**)
- Python dictionary is a powerful **data structure**
 - Treat dictionary as a Python **built-in database**
- The Python dictionary works the same way as the actual dictionary –
you look up a key and get its value(s)
- Dictionary organizes information for fast retrieval
- Very efficient

Dictionaries Store Key-Value Pairs

```
data = {'Tom': 172, 'Joe': 181, 'Mary': 168, 'Alice': 159}
print(data)
# {'Joe': 181, 'Mary': 168, 'Tom': 172, 'Alice': 159}
print(data['cat'])          # KeyError: 'cat'
print('cat' in data)        # False
print('Alice' in data)     # True
```

- The **key** is the lookup (e.g. 'Mary' here)
- The **value** is the corresponding value (e.g. 168 here)
- We say that a dictionary **maps from keys to values**
- **Dictionaries do not care about order (i.e., unordered)**
- Python complains with a **KeyError** if the key doesn't exist!
- You can check if a key exists first using **in**

Add / Update / Remove Key-Value pairs

```
data = {}                # start with an empty dictionary
data['Joe'] = 181
data['Alice'] = 159
data['Sue'] = 165
print(data)              # {'Sue': 165, 'Joe': 181, 'Alice': 159}
del data['Joe']           # remove a key-value pair by del statement
data['Alice'] = 163       # update a key's value by assignment
print(data)              # {'Sue': 165, 'Alice': 163}
# use for loop to traverse the keys on the dictionary
for key in data:
    print(key, "maps to", data[key])
    # get the values using the key
# Alice maps to 163
# Sue maps to 165
# notice that they still are not in order!
```

Dictionary

- A Python dictionary is an extremely useful data storage construct for organizing information in pairs (**key**: **value**)
- The association of a key with a value is called a **key-value pair** or an **item**
- **Key** must be **immutable** type (`str`, `int`, `float`, `tuple`)
 - Any **key** is unique
 - A **key** can associate with one or more values in a list
- **Value** can be any Python data type
- **Dictionaries are mutable** → dictionaries cannot be used as keys, but they **can be used as values**

Dictionary

- Similar to list, dictionaries can be changed, be shrunked, and grown when executing
 - Add / Update / Remove Key-Value pairs
- Dictionaries can be contained in lists and vice versa

```
x = { "John": [170, 75], "Mary": [160, 50] }  
y = [ { 1: "one", 2: "two" }, { "+": "add", "-": "subtract" } ]
```
- What's the difference between list and dictionary?

	List	Dictionary
Container	Order	Unordered
Access items	by position	by key

Access Items in Dictionary

```
dictionary = {key1:value1, key2:value2, ...}
```

- Direct access: by `key` index in `[]`
- `get (key)`: **return `None` if no such key exists**
- `keys ()`: return the list of keys (i.e., get all keys)
- `values ()`: return the list of values (i.e., get all values)

```
data = {'Name': 'Zara', 'Age': 7, 'Gender': 'F'}
print(data['Age'])           # 7
print(data.get('Age'))       # 7
print(data.get('City'))      # None
print(data.keys())           # dict_keys(['Age', 'Name', 'Gender'])
print(list(data.keys()))     # ['Gender', 'Name', 'Age']
print(data.values())         # dict_values(['F', 'Zara', 7])
print(list(data.values()))   # ['F', 'Zara', 7]
```

Access Items in Dictionary

- `items()`: returns a list of all **items**
Each **item** is a (key, value) pair or **tuple**
 - Most efficient way to examine all the key-value data
- Use **for** loop to traverse through the dictionary

```
data = {'Name': 'Zara', 'Age': 7, 'Gender': 'F'}  
for key in data: # the keys are in a random order  
    print(key, " - ", data[key])  
for key in data.keys():  
    print(key, " - ", data[key])  
for key, value in data.items():  
    print(key, " - ", value)  
print(list(data.items())) # [('Name', 'Zara'), ('Age', 7), ('Gender', 'F')]  
for key in sorted(data.keys()): # loop over the keys in sorted order  
    print(key, " - ", data[key])
```

Gender - F
Name - Zara
Age - 7

Age - 7
Gender - F
Name - Zara

Update a Dictionary

- Add a new entry or item (i.e., a key-value pair) → only if the key is not already in dictionary (unique keys)
- Modify an existing entry → if the key is already present, overwrites the previous value
- `del`: delete an item from a dictionary by key
- `clear()`: delete all items from a dictionary

```
data = {'Name': 'Zara', 'Age': 7, 'Gender': 'F'}
data['Age'] = 8 # update existing entry
data['School'] = "DPS School" # add new entry
print(data) # {'School': 'DPS School', 'Gender': 'F', 'Age': 8, 'Name': 'Zara'}
del data['Gender']
print(data) # {'School': 'DPS School', 'Age': 8, 'Name': 'Zara'}
data.clear()
print(data) # {}
```

Dictionary Operations

- `len()` : return the number of key-value pairs
- `in` operator: tell you whether something appears as a **key** in the dictionary
- `copy()` : make a copy of the dictionary

```
eng2sp = {'one': 'uno', 'two': 'dos', 'three': 'tres'}
print(len(eng2sp))           # 3
print('one' in eng2sp)      # True
print('uno' in eng2sp)      # False
copyDict = eng2sp.copy()
print(copyDict)
# {'three': 'tres', 'one': 'uno', 'two': 'dos'}
key = 'one'
if key in eng2sp:
    print(eng2sp[key])      # uno
```


Combine two Dictionaries / Build a Dictionary

- `update()` : combine two dictionaries
by using `update()` of the primary dictionary
 - Merge existing elements if they conflict
- `dict()` : build dictionary from sequences of key-value pairs
 - When the keys are simple strings, it is sometimes easier to specify pairs using keyword arguments

```
f1 = {'apples': 1, 'oranges': 3, 'pears': 2}
f2 = {'pears': 4, 'grapes': 5}
f1.update(f2)
print(f1) # {'oranges': 3, 'pears': 4, 'grapes': 5, 'apples': 1}
d1 = dict([('Sam', 171), ('Mary', 168), ('Jack', 180)])
print(d1) # {'Sam': 171, 'Mary': 168, 'Jack': 180}
d2 = dict(Sam = 171, Mary = 168, Jack = 180)
print(d2) # {'Sam': 171, 'Mary': 168, 'Jack': 180}
```

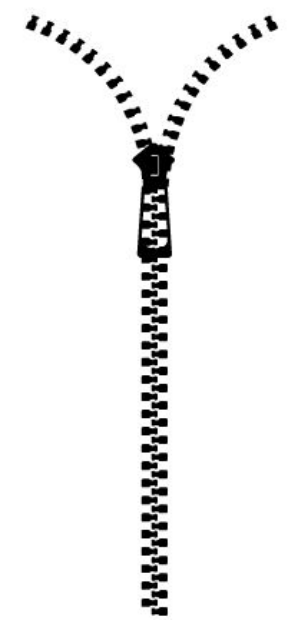
More on Dictionary

- **Tuple** as keys are ok! (keys are immutable)
- **Lists** as values are ok! (values can be anything)
- If you try to access a key which doesn't exist, you will get an error message
 - You can prevent this by using the **in** operator

```
data = {(160,99): "Unhealthy", (180,70): "Healthy"}
print(data[(180,70)])           # Healthy
grades = {'Fall-2016':[80,95,77], 'Fall-2017':[98,93]}
print(grades['Fall-2016'])      # [80, 95, 77]

words = {"house":"Haus", "cat":"Katze"}
if "car" in words:
    print(words["car"])         # nothing printed
```

Dictionaries from Lists



- Use `zip()` then `dict()` to combine the two lists into a dictionary
- `zip()`: Pair the elements of equal-length sequences into a list of tuples

```
dishes = ["pizza", "sauerkraut", "paella", "Hamburger"]
countries = ["Italy", "Germany", "Spain", "USA"]
country_specialities = zip(countries, dishes)
print(list(country_specialities))
# [('Italy', 'pizza'), ('Germany', 'sauerkraut'), ('Spain', 'paella'), ('USA', 'Hamburger')]
country_specialities_dict = dict(zip(countries, dishes))
print(country_specialities_dict)
# {'Germany': 'sauerkraut', 'USA': 'Hamburger', 'Italy': 'pizza', 'Spain': 'paella'}
```

Dictionaries from Lists

- What happens if the two lists do not have the same number of elements?
→ Superfluous elements will not be used!

```
dishes = ["pizza", "sauerkraut", "paella", "Hamburger"]
countries = ["Italy", "Germany", "Spain", "USA",
"Switzerland"]
country_specialities = zip(countries, dishes)
country_specialities_dict = dict(country_specialities)
print(country_specialities_dict)
# {'Germany': 'sauerkraut', 'Spain': 'paella', 'Italy':
'pizza', 'USA': 'Hamburger'}
# Sorry... The food specialty of Switzerland is not included
in today's class!
```

Tuple

- Fixed length, **immutable** sequence of objects
 - Scalars, sequences, dictionaries
- **Packing**: create using a comma-separated list of objects, **with or without parentheses**
- **Unpacking**: assign objects in a tuple to different variables
- Provide a quick way to swap variables' values

```
# packing
stock = ("Google", 100, 490.1)
stock = "Google", 100, 490.1
print(stock[0]) # "Google"
values = (170, 65, "M", 500)
values = 170, 65, "M", 500
print(values[0]) # 170
print(values[2]) # "M"
print(values[1:3]) # (65, "M")

# unpacking
h, w, g, s = values
print(h, g, s) # 170 "M" 500
h, w, g, s = 165, 48, "F", 550
x, y = 5, 10
x, y = y, x
print(x, y) # 10, 5
```

Tuple

- Tuples can contain different types of objects
- Tuples are **immutable**
- Can cast other objects into tuples using the **tuple** function
 - Can cast other objects into lists using the **list** function
- Check whether an object is in the tuple by **in**

```
myTuple = ([1,2], [2,3], "Hi")  
myTuple[0] = [5,6]
```

```
TypeError: 'tuple' object does  
not support item assignment
```

```
myTuple[0][1] = 9  
print(myTuple)  
# ([1,9], [2,3], "Hi")
```

```
vals = [6, 2, 3]  
tup = tuple(vals) # (6, 2, 3)  
lst = list(tup)   # [6, 2, 3]  
3 in lst # True  
"Hello" in myTuple # False
```

Tuple are Immutable

- Unlike a `list`, once you create a `tuple`, you **cannot alter its contents** – similar to a `string`

```
x = [9, 8, 7]    # list
```

```
x[2] = 6
```

```
print(x) # [9, 8, 6]
```

```
y = "ABCDE"     # string
```

```
y[2] = "D"
```

```
# 'str' object does not support item assignment
```

```
z = (5, 4, 3)    # tuple
```

```
z[2] = 0
```

```
# 'tuple' object does not support item assignment
```

Tuple

- Tuples with a single element needs to have comma (,)
- `len()` returns the number of elements
- `+` is for concatenation of two tuples

```
myTuple = ("Hello")
type(myTuple) # <class 'str'>
print(myTuple) # Hello
myTuple = ("Hello", )
print(myTuple) # ("Hello",)
type(myTuple) # <class 'tuple'>

tup1 = ("Tom", 175, 65)
len(tup1) # 3
tup2 = ("Mary", 163, 47)
tup = tup1 + tup2
print(tup)
# ("Tom", 175, 65, "Mary", 163, 47)
```


Tuple

- Tuples usually contain **heterogeneous** data while lists usually contain **homogeneous** data
- **Allow functions to return multiple values**

```
colors = [  
    ("red", "#FF0000", (255, 0, 0)),  
    ("green", "#00FF00", (0, 255, 0)),  
    ("blue", "#0000FF", (0, 0, 255))  
] # common to see this form  
def bounds(values)  
    low = min(values)  
    high = max(values)  
    return (low, high)  
print(bounds([3, -5, 9, 4, 17, 0])) # (-5, 17)  
least, greatest = bounds([3, -5, 9, 4, 17, 0])  
print(least, greatest) # -5 17
```

Tuple

- Tuples often used in loops
- Some functions used in lists do not support tuples

```
pairs = ((1, 10), (2, 20), (3, 30), (4, 40))
for p in pairs:
    print(p[0] + p[1], end=" ")    # 11 22 33 44
for (low, high) in pairs:
    print(low + high, end=" ")    # 11 22 33 44
x = (3, 2, 1)
x.sort()
# AttributeError: 'tuple' object has no attribute 'sort'
x.append(5)
# AttributeError: 'tuple' object has no attribute 'append'
x.reverse()
# AttributeError: 'tuple' object has no attribute 'reverse'
```

Built-in Functions for Tuple

```
x = (1, 2, 3, 4, 5, 2, 4, 6, 2, 9, 7)
print(x.count(2))    # 3
print(x.index(2))    # 1
print(len(x))        # 11
print(max(x))        # 9
print(min(x))        # 1
print(sorted(x))     # [1, 2, 2, 2, 3, 4, 4, 5, 6, 7, 9]
print(sum(x))        # 45

y = (5, 2, "Hello", 7.5, (1,3), [8,9,0])
# enumerate(): return an enumerate object. It contains the index and
# value of all the items of tuple as pairs.
for item in enumerate(y):
    print(item)
# (0, 5)
# (1, 2)
# (2, 'Hello')
# (3, 7.5)
# (4, (1, 3))
# (5, [8, 9, 0])
```

Notes for Tuples

- Once created, never changes
 - If you have data that doesn't change, implementing it as tuple will guarantee that it remains write-protected
- A programmer decides its values and length
- No operator or built-in function to modify tuples
- Lists slower but more powerful than tuples
 - Tuples are more efficient
- Convert between tuples and lists using `list()` and `tuple()` functions

Set

- **Set { }:** mutable, unordered collection of unique elements
- Use the `set()` function to cast a list to a set of unique values
- **Items in a set must be immutable**
 - int, float, str, and tuple can be members of a set
 - list, dict, and set themselves cannot

```
basket = {'apple', 'orange',  
          'apple', 'pear', 'orange',  
          'banana'}  
print(basket)  
# {'orange', 'banana', 'pear',  
#  'apple'}  
basket2 = ['apple', 'orange',  
           'apple', 'pear', 'orange',  
           'banana']  
fruit = set(basket2)  
  
a = set("abracadabra")  
print(a)  
# {'a', 'r', 'b', 'c', 'd'}  
mySet = {[1,3,5], [4,6]}  
# TypeError: unhashable type:  
# 'list'
```

Set

- Sets have **no indices** (unordered)
- Sets contain **no duplicate objects** (unique)
- Set is **mutable** by using **set operations**
 - **+** : Error! No + for sets
 - **-** : set subtraction
 - **|** : set union (or)
 - **&** : set intersection (and)
 - **^** : set “exclusive or”
 - **in** : membership testing

```
x = set('abracadabra')
y = set('alacazam')
print(x) # {'c', 'a', 'r', 'b', 'd'}
print(y) # {'c', 'a', 'm', 'z', 'l'}
print(x[2]) # TypeError: 'set'
            # object does not support indexing
print(x + y)
# TypeError: unsupported operand
# type(s) for +: 'set' and 'set'
print(x - y) # {'r', 'b', 'd'}
print(x | y) # {'m', 'l', 'b',
              # 'c', 'z', 'a', 'r', 'd'}
print(x & y) # {'a', 'c'}
print(x ^ y) # {'m', 'b', 'd',
              # 'z', 'r', 'l'}
print("c" in x) # True
print(3 in y)   # False
```

Built-in Functions for Sets

- `add(e)`: add an element `e`
- Remove an element `e`
 - `discard(e)`: no error if element `e` is not there
 - `remove(e)`: error if element `e` is not there
- `set1.issubset(set2)`
- `set1.issuperset(set2)`
- `len(myset)`
- Set operations from math
 - `set1.union(set2)`
 - `set1.intersection(set2)`
 - `set1.difference(set2)`

```
x = {1, 2, 3, 5, 2, 6, 2}
y = {3, 5, 7, 8, 1}
z = {1, 2}
x.add(9)
print(x)      # {1, 2, 3, 5, 6, 9}
x.remove(6)
print(x)      # {1, 2, 3, 5, 9}
x.remove(0)    # KeyError: 0
x.discard(0)   # no error
print(z.issubset(x)) # True
print(x.issuperset(z)) # True
print(z.issubset(y)) # False
print(len(x))    # 5
w = y.union(z)
print(w) # {1, 2, 3, 5, 7, 8}
u = x.intersection(y)
print(u) # {1, 3, 5}
```




AI如果是輛衝往未來的火車，機器學習與深度學習就是它的引擎，而引擎是由程式語言組裝而成，數據及其分析是它的燃料。多數人還沒有登上這台火車，停留在新聞報導或科幻小說上看到的AI，但每天的生活卻早已被這些AI驅動的龐大系統所影響。